

Practical7: Neural Network implementaion on Iris dataset

ID : 17CE049

DataSet : Bike Sharing Data set <https://archive.ics.uci.edu/ml/datasets/bike+sharing+dataset>
(<https://archive.ics.uci.edu/ml/datasets/bike+sharing+dataset>)

Task : In this practical, we will build our first neural network and use it to predict daily bike rental ridership.

In [14]:

```
%matplotlib inline
%load_ext autoreload
%autoreload 2
%config InlineBackend.figure_format = 'retina'

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

The autoreload extension is already loaded. To reload it, use:
%reload_ext autoreload

Load and prepare the data

In [15]:

```
data_path = 'Bike-Sharing-Dataset/hour.csv'

rides = pd.read_csv(data_path)
```

In [16]:

```
rides.head()
```

Out[16]:

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingday	weathersit	temp	atemp
0	1	2011-01-01	1	0	1	0	0	6	0	1	0.24	0.2
1	2	2011-01-01	1	0	1	1	0	6	0	1	0.22	0.2
2	3	2011-01-01	1	0	1	2	0	6	0	1	0.22	0.2
3	4	2011-01-01	1	0	1	3	0	6	0	1	0.24	0.2
4	5	2011-01-01	1	0	1	4	0	6	0	1	0.24	0.2

Exploring Data

This dataset has the number of riders for each hour of each day from January 1 2011 to December 31 2012. The number of riders is split between casual and registered, summed up in the `cnt` column. You can see the first few rows of the data above.

Below is a plot showing the number of bike riders over the first 10 days or so in the data set. (Some days don't have exactly 24 entries in the data set, so it's not exactly 10 days.)

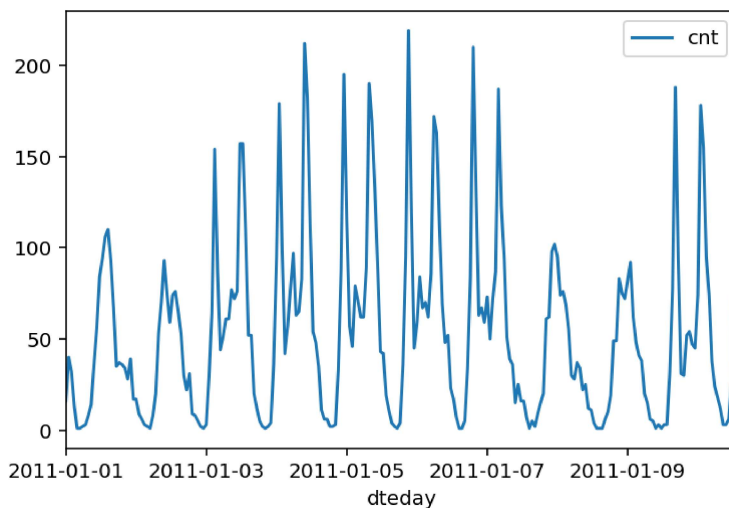
You can see the hourly rentals here. The weekends have lower over all ridership and there are spikes when people are biking to and from work during the week. Looking at the data above, we also have information about temperature, humidity, and windspeed, all of these likely affecting the number of riders.

In [17]:

```
rides[:24*10].plot(x='dteday',y='cnt')
```

Out[17]:

<matplotlib.axes._subplots.AxesSubplot at 0x242369b84c8>



Dummy variables

Here we have some categorical variable like season, weather, month. To include these in our model, we'll need to make binary dummy variables.

In [19]:

```

dummy_variables = ['season', 'weathersit', 'mnth', 'hr', 'weekday']
for each in dummy_variables:
    dummies = pd.get_dummies(rides[each], prefix=each, drop_first=False)
    rides = pd.concat([rides, dummies], axis=1)

fields_to_drop = ['instant', 'dteday', 'season', 'weathersit',
                  'mnth', 'hr', 'weekday', 'atemp', 'workingday']

data = rides.drop(fields_to_drop, axis=1)
data.head()

```

Out[19]:

	yr	holiday	temp	hum	windspeed	casual	registered	cnt	season_1	season_2	...	hr_21
0	0	0	0.24	0.81	0.0	3	13	16	1	0	...	0
1	0	0	0.22	0.80	0.0	8	32	40	1	0	...	0
2	0	0	0.22	0.80	0.0	5	27	32	1	0	...	0
3	0	0	0.24	0.75	0.0	3	10	13	1	0	...	0
4	0	0	0.24	0.75	0.0	0	1	1	1	0	...	0

5 rows × 59 columns

Scaling target variables

For training we'll standardize each continuous variables. we'll shift and scale the variables such that they have zero mean and standard deviation of 1.

and we'll save this scaling factor to use it later to go backward.

In [22]:

```

cont_features = ['casual', 'registered', 'cnt', 'temp', 'hum', 'windspeed']
# stored in dictionary to convert back
scaled_features = {}
for each in cont_features:
    mean, std = data[each].mean(), data[each].std()
    scaled_features[each] = [mean, std]
    data.loc[:, each] = (data[each] - mean) / std

```

Splitting the data into training, testing, and validation sets

In [46]:

```
#here we will take last 21 days as test set

test_data = data[-21*24:]

data = data[:-21*24]

target_fields = ['cnt', 'casual', 'registered']
features, targets = data.drop(target_fields, axis=1), data[target_fields]
test_features, test_targets = np.array(test_data.drop(target_fields, axis=1)), np.array(test_data[target_fields])
```

Now split training data in training and validation set

In [47]:

```
# Hold out the last 60 days or so of the remaining data as a validation set
train_features, train_targets = np.array(features[:-60*24]), np.array(targets[:-60*24])
val_features, val_targets = np.array(features[-60*24:]), np.array(targets[-60*24:])
```

Create Data Loader

In [49]:

```
from torch.utils.data import Dataset, DataLoader
class RegressionDataset(Dataset):

    def __init__(self, X_data, y_data):
        self.X_data = X_data
        self.y_data = y_data

    def __getitem__(self, index):
        return self.X_data[index], self.y_data[index]

    def __len__(self):
        return len(self.X_data)

train_dataset = RegressionDataset(torch.from_numpy(train_features).float(), torch.from_numpy(train_targets).float())
val_dataset = RegressionDataset(torch.from_numpy(val_features).float(), torch.from_numpy(val_targets).float())
test_dataset = RegressionDataset(torch.from_numpy(test_features).float(), torch.from_numpy(test_targets).float())
```

In [50]:

```
BATCH_SIZE = 128
train_loader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(dataset=val_dataset, batch_size=1)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)
```

Create and Train Neural Network

In [28]:

```
import torch
from torch import nn
import torch.nn.functional as F
from torch import optim
```

In [57]:

```
class Regressor(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(train_features.shape[1], 25)
        self.fc2 = nn.Linear(25, 10)
        self.fc3 = nn.Linear(10, 1)

    def forward(self, x):
        x = x.view(x.shape[0], -1)

        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

model = Regressor()
model.to(device)
print(model)

criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
```

```
Regressor(
  (fc1): Linear(in_features=56, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=10, bias=True)
  (fc3): Linear(in_features=10, out_features=1, bias=True)
)
```

In [58]:

```

epochs = 1000
train_losses, test_losses = [], []

for e in range(epochs):

    train_epoch_loss = 0

    model.train()
    for X_train_batch, y_train_batch in train_loader:
        X_train_batch, y_train_batch = X_train_batch.to(device), y_train_batch.to(device)

        optimizer.zero_grad()

        output = model(X_train_batch)
        loss = criterion(output, y_train_batch.unsqueeze(1))
        loss.backward()
        optimizer.step()
        train_epoch_loss += loss.item()

    with torch.no_grad():
        val_epoch_loss = 0
        model.eval()
        for X_val_batch, y_val_batch in val_loader:
            X_val_batch, y_val_batch = X_val_batch.to(device), y_val_batch.to(device)
            log_ps = model(X_val_batch)
            val_loss = criterion(log_ps, y_val_batch.unsqueeze(1))
            val_epoch_loss += val_loss.item()

    model.train()

    train_losses.append(train_epoch_loss/len(train_loader))
    test_losses.append(val_epoch_loss/len(val_loader))

    if(e%100==99 or e==0):
        print("Epoch: {}/{}.. ".format(e+1, epochs),
              "Training Loss: {:.3f}.. ".format(train_epoch_loss/len(train_loader)),
              "Test Loss: {:.3f}.. ".format(val_epoch_loss/len(val_loader)))

```

```

Epoch: 1/1000.. Training Loss: 0.542.. Test Loss: 0.817..
Epoch: 100/1000.. Training Loss: 0.034.. Test Loss: 0.159..
Epoch: 200/1000.. Training Loss: 0.031.. Test Loss: 0.128..
Epoch: 300/1000.. Training Loss: 0.030.. Test Loss: 0.137..
Epoch: 400/1000.. Training Loss: 0.029.. Test Loss: 0.135..
Epoch: 500/1000.. Training Loss: 0.029.. Test Loss: 0.138..
Epoch: 600/1000.. Training Loss: 0.028.. Test Loss: 0.164..
Epoch: 700/1000.. Training Loss: 0.027.. Test Loss: 0.172..
Epoch: 800/1000.. Training Loss: 0.027.. Test Loss: 0.173..
Epoch: 900/1000.. Training Loss: 0.027.. Test Loss: 0.176..
Epoch: 1000/1000.. Training Loss: 0.026.. Test Loss: 0.177..

```

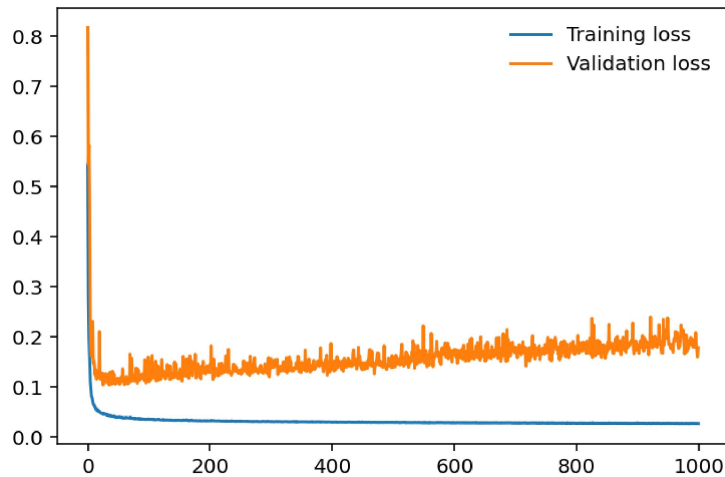
Plotting Training and Validation Loss

In [59]:

```
plt.plot(train_losses, label='Training loss')
plt.plot(test_losses, label='Validation loss')
plt.legend(frameon=False)
```

Out[59]:

<matplotlib.legend.Legend at 0x242045e3c88>



Testing Model

In [67]:

```
y_pred_list = []
with torch.no_grad():
    model.eval()
    for X_batch, _ in test_loader:
        X_batch = X_batch.to(device)
        y_test_pred = model(X_batch)
        y_pred_list.append(y_test_pred.cpu().numpy())
y_pred_list = [a.squeeze().tolist() for a in y_pred_list]
```

In [68]:

```
len(y_pred_list)
```

Out[68]:

504

In [69]:

```

from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(test_targets.reshape(-1,1), y_pred_list)
r_square = r2_score(test_targets.reshape(-1,1), y_pred_list)
print(f"Mean Squared Error :{mse}")
print(f"R^2 :{r_square}")

```

Mean Squared Error :0.264956959228802

R^2 :0.7492481801868556

Plotting Prediction With Original Values

In [73]:

```

fig, ax = plt.subplots(figsize=(8,4))

mean, std = scaled_features['cnt']
predictions = [i*std + mean for i in y_pred_list]
ax.plot(predictions, label='Prediction')
ax.plot([i*std + mean for i in test_targets], label='Data')
ax.set_xlim(right=len(predictions))
ax.legend()

dates = pd.to_datetime(rides.ix[test_data.index]['dteday'])
dates = dates.apply(lambda d: d.strftime('%b %d'))
ax.set_xticks(np.arange(len(dates))[12::24])
_ = ax.set_xticklabels(dates[12::24], rotation=45)

```

C:\Users\VATSAL\Anaconda3\lib\site-packages\ipykernel_launcher.py:10: Future Warning:

.ix is deprecated. Please use
.loc for label based indexing or
.iloc for positional indexing

See the documentation here:

http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#ix-indexer-is-deprecated (http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#ix-indexer-is-deprecated)

Remove the CWD from sys.path while we load stuff.

